

baldurs gate 3 hj a noite?

Unioeste

Amós, Carlos e Nikolas

9 de setembro de 2025

Índice

1	Strings	2	4.1	Centro de Árvore	11
1.1	Manacher	2	4.2	Coin Collector	11
1.2	StringStream	3	4.3	Divisor Analysis	12
2	Estrutura de Dados	3	4.4	Hungaro	13
2.1	BIT	3	4.5	Intervalo da String Palíndromico	14
2.2	SegTree Add	3	4.6	Inversões	14
2.3	SegTree Bits	4	4.7	Kadane	14
2.4	SegTree Lazy	4	4.8	Number of Pairs (GCD e LCM)	14
2.5	SegTree Update	5	4.9	Path Queries 2	16
2.6	Sparse Table	5	4.10	Steiner Tree	17
3	Primitivas	6	4.11	Subtree Queries	18
3.1	Aritmética Modular	6	5	Matemática	19
3.2	Geometria Double	6	5.1	Combinatoria	19
3.3	Geometria Int	9	5.2	EV Máximo	20
4	Problemas	11	5.3	Exponenciação Rápida	20
			5.4	Fatoração Prima	20
			5.5	GCD e LCM	20
			5.6	Phi Function	21

5.7	Segmented Sieve	21	48b Nao e eficiente pois utiliza substr O(size)
5.8	Sieve	21	c4c */
5.9	Smallest Prime Factor	21	
6	DP	22	fd3 vector<int> manacher_odd(string s) {
6.1	Edit Distance	22	163 int n = s.size();
6.2	LCS	22	0cb s = "\$" + s + "^";
7	Grafo	22	1a9 vector<int> p(n + 2);
7.1	Bridge Tree	22	187 int l = 0, r = 1;
7.2	Dijkstra	23	78a for(int i = 1; i <= n; i++) {
7.3	DSU	23	a1d p[i] = min(r - i, p[l + (r - i)]);
7.4	HLD - aresta	23	ce7 while(s[i - p[i]] == s[i + p[i]]) {
7.5	Kosaraju	24	fbf p[i]++;
7.6	LCA Sparse Table	25	dc2 }
7.7	Minimum Spanning Tree	25	03e if(i + p[i] > r) {
7.8	Pontos de Articulação	26	26d l = i - p[i];
7.9	Toposort	26	a69 r = i + p[i];
8	Extra	27	4cc }
8.1	debug.cpp	27	283 }
8.2	Template.cpp	30	fa1 return vector<int>(begin(p) + 1, end(p) - 1);
1	Strings		08d }
1.1	Manacher		4e5 vector<int> manacher(string s) {
			78f string t;
			7e7 for(auto c: s) {
			870 t += string("#") + c;
			478 }
			9cc auto res = manacher_odd(t + "#");
			b50 return res;
			cdc }
			0dd signed main() {
			5d6 winton;
			ac0 string s;
			a18 cin >> s;
			bf9 vector<int> mnc = manacher(s);
			35c vector<pair<int,int>> palindromes;
			63e for (int i = 0; i < mnc.size(); i++){
			1e8 if (mnc[i]-1 > 0){
			8ad palindromes.push_back({mnc[i]-1,i}); //guarda os
			palindromes com {tamanho, centro}
			ad1 }
			b1c }
			668 sort(rall(palindromes)); //ordena pelos maiores (opcional ne)
			603 for (auto &[size, centro] : palindromes){
			/*
			8fa Manacher para achar todos os palindromes e depois reconstruir eles

```

04c      string pos;
3f0      int st = ((centro - size)/2);
ab5      pos = s.substr(st, size); //gera a substring daquele
        palindrome
2dc    }
bf9 }

```

1.2 StringStream

```

141 vector<string> split_sentence(string s){
942     stringstream ss(s);

6de     string word;

622     vector<string> v;

dac     while(getline(ss, word, ' ')){
e6c         v.push_back(word);
8e0     }
6dc     return v;
c7e }

```

2 Estrutura de Dados

2.1 BIT

```

// BIT de xor 1-based
// faz o xor no intervalo
//
// Complexidades:
// build - O(n)
// update - O(log(n))
// query - O(log(n))

```

```

8eb struct Bit {
1a8     int n;
116     vector<int> bit;
e86     Bit(int _n=0) : n(_n), bit(n + 1) {}
cf6     void update(int i, int x) { // xor x na posicao i
f0e         for (i; i <= n; i += i & -i) bit[i] ^= x;
2a0     }
cdb     int pref(int i) { // xor [0, i]

```

```

7c9         int ret = 0;
653         for (i; i; i -= i & -i) ret ^= bit[i];
edf         return ret;
7d1     }
9e3     int query(int l, int r) { // xor [l, r]
da4         debug(pref(r), pref(l-1));
872         return pref(r) ^ pref(l - 1);
10d     }
3d4 };

```

2.2 SegTree Add

```

//Get o caminho

```

```

7e4 tree[4*MAX];

```

```

037 void build(int node, int l, int r){
893     if (l == r){
c91         tree[node] = 0; //alguma coisa
505         return;
447     }
ee4     int m = (l+r)/2;
435     build(node*2, l, m);
4c9     build(node*2+1, m+1, r);
452 }

```

```

10d int get(int node, int l, int r, int i){
a15     if (l == r) return tree[node];
ee4     int m = (l+r)/2;
7c4     int res;
979     if (i > m) res = get(node*2+1, m+1, r, i);
d04     else res = get(node*2, l, m, i);
eac     return res + tree[node];
ca6 }

```

```

506 void add(int node, int l, int r, int a, int b, int x){
1dd     if(b < l or r < a) return;
02b     if(a <= l && r <= b){
3c6         tree[node] += x;
505         return;
5cc     }
ee4     int m = (l+r)/2;
3da     add(2*node, l, m, a, b, x);
307     add(2*node+1, m+1, r, a, b, x);
616 }

```

```

5ed add(1,0,n-1, l, r, value)
1e7 get(1,0,n-1, index_of_search)

```

2.3 SegTree Bits

```

f20 si = vetor se diz se aquela posicao esta valida/funcionando
483 on = arvore que conta quantos estao ligados
b7d valid = arvore que conta quantos sao validos naquele intervalo
(exclusivo de um problema la)
007 toda operacao ele calcula os ligados com base em quantos tem
valido naquele intervalo - os desligados, ja que ta sempre
invertendo
c4c */
99a int n, tree[4*MAX], on[4*MAX], valid[4*MAX], si[MAX];
1d3 bool lazy[4*MAX];

037 void build(int node, int l, int r){
48c     lazy[node] = false;
893     if (l == r){
811         if (si[l] == -1){
809             on[node] = 0;
600             valid[node] = 0;
771         }
4e6         else{
a5b             valid[node] = 1;
342             on[node] = si[l];
4f9         }
505         return;
370     }
ee4     int m = (l+r)/2;
435     build(node*2, l, m);
4c9     build(node*2+1, m+1, r);
931     valid[node] = valid[node*2] + valid[node*2+1];
be9     on[node] = on[node*2] + on[node*2+1];
0f7 }

885 void flip(int node){
890     on[node] = valid[node] - on[node];
006     lazy[node] = !lazy[node];
c10 }

7ec void prop(int node){
996     if (lazy[node]){
1c2         flip(node*2);
f5e         flip(node*2+1);
48c         lazy[node] = false;
a99     }

```

```

e8d }

938 void update(int node, int l, int r, int a, int b){
1dd     if(b < l or r < a) return;
02b     if(a <= l && r <= b){
f7b         flip(node);
505         return;
b4b     }
dcf     prop(node);
ee4     int m = (l+r)/2;
325     update(2*node, l, m, a, b);
cf4     update(2*node+1, m+1, r, a, b);
be9     on[node] = on[node*2] + on[node*2+1];
031 }

```

2.4 SegTree Lazy

```

// Recursiva com Lazy Propagation
// Query: soma do range [a, b]
// Update: soma x em cada elemento do range [a, b]
//
// Complexidades:
// build - O(n)
// query - O(log(n))
// update - O(log(n))
//
// build (n, vector)
// query (int a, int b)
// update (int a, int b, int x)
// se for de multiplicacao mudar o lazy pra 1 e tirar o lazy * (r-l+1)

fb1 namespace SegTree {
d18     int tree[4*MAX], lazy[4*MAX];
052     int n, *v;

34c     int build(int node=1, int l=0, int r=n-1) {
131         lazy[node] = 0;
4e3         if (l == r) return tree[node] = v[l];
ee4         int m = (l+r)/2;
19c         return tree[node] = build(2*node, l, m) + build(2*node+1,
m+1, r);
851     }
0d8     void build(int n2, int* v2) {
680         n = n2, v = v2;
6f2         build();
acb     }

```

```

dd7 void prop(int node, int l, int r) {
7eb     tree[node] += lazy[node]*(r-l+1);
17c     if (l != r) lazy[2*node] += lazy[node], lazy[2*node+1] +=
        lazy[node];
131     lazy[node] = 0;
6eb }
a2c int query(int a, int b, int node=1, int l=0, int r=n-1) {
020     prop(node, l, r);
e09     if (a <= l and r <= b) return tree[node];
786     if (b < l or r < a) return 0;
ee4     int m = (l+r)/2;
1db     return query(a, b, 2*node, l, m) + query(a, b, 2*node+1,
        m+1, r);
cf9 }
c96 int update(int a, int b, int x, int node=1, int l=0, int
        r=n-1) {
020     prop(node, l, r);
9a3     if (a <= l and r <= b) {
3bd         lazy[node] += x;
020         prop(node, l, r);
14d         return tree[node];
57d     }
ad9     if (b < l or r < a) return tree[node];
ee4     int m = (l+r)/2;
dcf     return tree[node] = update(a, b, x, 2*node, l, m) +
        update(a, b, x, 2*node+1, m+1, r);
eca }
d1a }

```

2.5 SegTree Update

```

// Recursiva sem Lazy Propagation
// Query: soma do range [a, b]
// Update: soma x em cada elemento do range [a, b]
//
fb1 namespace SegTree {
34a     int tree[4*MAX];
052     int n, *v;

34c     int build(int node=1, int l=0, int r=n-1) {
4e3         if (l == r) return tree[node] = v[l];
ee4         int m = (l+r)/2;
19c         return tree[node] = build(2*node, l, m) + build(2*node+1,
            m+1, r);
913     }

```

```

0d8 void build(int n2, int* v2) {
680     n = n2, v = v2;
6f2     build();
acb }

a2c int query(int a, int b, int node=1, int l=0, int r=n-1) {
786     if (b < l or r < a) return 0;
e09     if (a <= l and r <= b) return tree[node];
ee4     int m = (l+r)/2;
1db     return query(a, b, 2*node, l, m) + query(a, b, 2*node+1,
        m+1, r);
09b }

eb3 void update(int i, int x, int node=1, int l=0, int r=n-1) {
893     if (l == r) {
d6d         tree[node] = x;
505         return;
b74     }
ee4     int m = (l+r)/2;
3df     if (i <= m) update(i, x, 2*node, l, m);
0a9     else update(i, x, 2*node+1, m+1, r);
8fb     tree[node] = tree[2*node] + tree[2*node+1];
dad }
ce6 }

```

2.6 Sparse Table

```

// Resolve RMQ
//
// Complexidades:
// build - O(n log(n))
// query - O(1)

cca namespace sparse {
d28     int m[LOG+1][MAX], n;
61c     void build(int n2, int* v) {
1e3         n = n2;
78e         for (int i = 0; i < n; i++) m[0][i] = v[i];
a1c         for (int j = 1; (1<<j) <= n; j++) for (int i = 0; i+(1<<j)
            <= n; i++)
5d5             m[j][i] = min(m[j-1][i], m[j-1][i+(1<<(j-1))]);
cae     }
4ea     int query(int a, int b) {
ee5         int j = __builtin_clz(1) - __builtin_clz(b-a+1);
dc3         return min(m[j][a], m[j][b-(1<<j)+1]);
fba     }

```

```
3e1 }
```

3 Primitivas

3.1 Aritmética Modular

```
429 template<int p> struct mod_int {
007     int expo(int b, int e) {
856         int ret = 1;
c87         while (e) {
cad             if (e % 2) ret = ret * b % p;
9d2             e /= 2, b = b * b % p;
c42         }
edf         return ret;
170     }
572     int inv(int b) { return expo(b, p-2); }

4d7     using m = mod_int;
d93     int v;
fe0     mod_int() : v(0) {}
cde     mod_int(int v_) {
019         if (v_ >= p or v_ <= -p) v_ %= p;
bc6         if (v_ < 0) v_ += p;
2e7         v = v_;
e1d     }
74d     m& operator +=(const m& a) {
2fd         v += a.v;
ba5         if (v >= p) v -= p;
357         return *this;
c8b     }
eff     m& operator -=(const m& a) {
8b4         v -= a.v;
cc8         if (v < 0) v += p;
357         return *this;
f8d     }
4c4     operator *=(const m& a) {
830         v = v * int(a.v) % p; //tirar o int quando for usar define
int long long
357         return *this;
623     }
3f9     m& operator /=(const m& a) {
5d6         v = v * inv(a.v) % p;
357         return *this;
62d     }
```

```
d65     m operator -(){ return m(-v); }
e8f     m& operator ^=(int e) {
06d         if (e < 0) {
6e2             v = inv(v);
00c             e = -e;
275         }
284         v = expo(v, e);
// possível otimizacao:
// cuidado com 0^0
// v = expo(v, e%(p-1));
357         return *this;
b6d     }
423     bool operator ==(const m& a) { return v == a.v; }
69f     bool operator !=(const m& a) { return v != a.v; }

1c6     friend istream& operator >>(istream& in, m& a) {
9bf         int val; in >> val;
d48         a = m(val);
091         return in;
a6f     }
44f     friend ostream& operator <<(ostream& out, m a) {
5a0         return out << a.v;
214     }
399     friend m operator +(m a, m b) { return a += b; }
f9e     friend m operator -(m a, m b) { return a -= b; }
9c1     friend m operator *(m a, m b) { return a *= b; }
51b     friend m operator /(m a, m b) { return a /= b; }
937     friend m operator ^(m a, int e) { return a ^= e; }
1a1 };

b49 typedef mod_int<(int)MOD> mint;

e9d int modiv(int a, int b){
659     return (((a % MOD ) * (fastExpo(b, MOD-2) % MOD)) % MOD);
e50 }
```

3.2 Geometria Double

```
c83 typedef double ld;
e3b const ld DINF = 1e18;
43a const ld pi = acos(-1.0);
107 const ld eps = 1e-9;

b32 #define sq(x) ((x)*(x))

d97 bool eq(ld a, ld b) {
```

```

fe4     return fabs(a - b) <= eps;
dac }

be5 struct point { // ponto
c1e     ld x, y;
018     point(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
be2     bool operator < (const point p) const {
059         if (!eq(x, p.x)) return x < p.x;
f98         if (!eq(y, p.y)) return y < p.y;
bb3         return 0;
ab8     }
418     bool operator == (const point p) const {
ed0         return eq(x, p.x) and eq(y, p.y);
4a1     }
460     point operator + (const point p) const { return point(x+p.x,
y+p.y); }
a2f     point operator - (const point p) const { return point(x-p.x,
y-p.y); }
807     point operator * (const ld c) const { return point(x*c , y*c
); }
669     point operator / (const ld c) const { return point(x/c , y/c
); }
986     ld operator * (const point p) const { return x*p.x + y*p.y; }
d94     ld operator ^ (const point p) const { return x*p.y - y*p.x; }
b2b     friend istream& operator >> (istream& in, point& p) {
e37         return in >> p.x >> p.y;
d70     }
70a };

b3a struct line { // reta
b1f     point p, q;
0d6     line() {}
50a     line(point p_, point q_) : p(p_), q(q_) {}
8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }
ebb };

ee4 ld dist(point p, point q) { // distancia
5f3     return hypot(p.y - q.y, p.x - q.x);
23c }

06b ld dist2(point p, point q) { // quadrado da distancia
f24     return sq(p.x - q.x) + sq(p.y - q.y);
2ef }

f5c ld norm(point v) { // norma do vetor

```

```

923     return dist(point(0, 0), v);
568 }

0cf ld angle(point v) { // angulo do vetor com o eixo x
587     ld ang = atan2(v.y, v.x);
6f8     if (ang < 0) ang += 2*pi;
19c     return ang;
8c4 }

3ca ld sarea(point p, point q, point r) { // area com sinal
606     return ((q-p)^(r-q))/2;
5b6 }

6df bool col(point p, point q, point r) { // se p, q e r sao colin.
e7d     return eq(sarea(p, q, r), 0);
20f }

c65 bool ccw(point p, point q, point r) { // se p, q, r sao ccw
fa7     return sarea(p, q, r) > eps;
20e }

4cf point rotate(point p, ld th) { // rotaciona o ponto th radianos
4f9     return point(p.x * cos(th) - p.y * sin(th), p.x * sin(th) +
p.y * cos(th));
828 }

5cf point rotate90(point p) { // rotaciona 90 graus
128     return point(-p.y, p.x);
c3b }

// retorna t tal que t*v pertence a reta r
849 ld get_t(point a, line r) {
ac5     return (r.p^r.q) / ((r.p-r.q)^a);
0cf }

edc bool isvert(line r) { // se r eh vertical
87d     return eq(r.p.x, r.q.x);
0fb }

9d8 point proj(point p, line r) { // projecao do ponto p na reta r
bea     if (r.p == r.q) return r.p;
97a     r.q = r.q - r.p; p = p - r.p;
ead     point proj = r.q * ((p*r.q) / (r.q*r.q));
2cd     return proj + r.p;
d37 }

```

```

5d2 bool isinseg(point p, line r) { // se p pertence ao seg de r
517     point a = r.p - p, b = r.q - p;
b04     return eq((a ^ b), 0) and (a * b) < eps;
da8 }

956 point inter(line r, line s) { // r inter s
b99     if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return point(DINF, DINF);
205     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
543     return r.q * get_t(r.q, s) + r.p;
6c0 }

676 bool interseg(line r, line s) { // se o seg de r intersecta o seg
de s
2b6     if (isinseg(r.p, s) or isinseg(r.q, s) or isinseg(s.p, r) or
isinseg(s.q, r)) return 1;
085     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and ccw(s.p,
s.q, r.p) != ccw(s.p, s.q, r.q);
359 }

9f2 ld linepoint(point p, line r) { // distancia do ponto a reta
89a     return 2 * abs(sarea(p, r.p, r.q)) / dist(r.p, r.q);
389 }

639 ld segpoint(line r, point p) { // distancia do ponto ao seg
73d     if ((r.q - r.p)*(p - r.p) < 0) return dist(r.p, p);
951     if ((r.p - r.q)*(p - r.q) < 0) return dist(r.q, p);
199     return linepoint(p, r);
e86 }

ccc ld segseg(point a, point b, point c, point d) {
102     line ab(a,b);
001     line cd(c,d);
05c     if (interseg(ab, cd)) return 0;
494     return min({segpoint(ab,c), segpoint(ab,d), segpoint(cd,a),
segpoint(cd,b)});
3d6 }

//linha que corta o segmento AB no meio perpendicularmente
a8a line mediatriz(point p, point q) {
da4     point mid = (p + q) / 2.0;
b5e     point vec = q - p;
5e7     point p_vec = rotate90(vec);
b03     return line(mid, mid + p_vec);
5b8 }

//bissetriz interna e externa
//bissetriz = linha que forma angulos iguais com outras duas linhas

```

```

28b pair<line, line> bissetriz(line r, line s) {
255     if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return {};
ba5     point i = inter(r,s);
1d9     point v1 = r.p - i;
4c0     point v2 = s.p - i;
27e     if (norm(v1) < eps) v1 = r.q - i;
672     if (norm(v2) < eps) v2 = s.q - i;
44a     v1 = v1 / norm(v1); v2 = v2 / norm(v2);
675     line interna = line(i, i + (v1 + v2));
8a6     line externa = line(i, i + (v1 - v2));
7fa     return {interna, externa};
d22 }

a0a point findcircle(point p, point q, point r){//acha o centro do
circulo que encosta nos 3 pontos
55f     if (col(p,q,r)) return point(DINF,DINF);
9b6     line mid1 = mediatriz(p,q);
f9a     line mid2 = mediatriz(q,r);
294     return inter(mid1, mid2);
d41 }

a39 ld polarea(vector<point> v){ // area do poligono
9c5     ld ret = 0;
f14     for (int i = 0; i < v.size(); i++){
d0f         ret += sarea(point(0, 0), v[i], v[(i + 1) % v.size()]);
8cb     }
d03     return abs(ret);
22a }

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
fb6 int inpol(vector<point>& v, point p) { // 0(n)
8de     int qt = 0;
f14     for (int i = 0; i < v.size(); i++) {
bda         if (p == v[i]) return 2;
6af         int j = (i+1)%v.size();
e38         if (eq(p.y, v[i].y) and eq(p.y, v[j].y)) {
97f             if ((v[i]-p)*(v[j]-p) < eps) return 2;
5e2             continue;
48b         }
388         bool baixo = v[i].y+eps < p.y;
464         if (baixo == (v[j].y+eps < p.y)) continue;
366         auto t = (p-v[i])^(v[j]-v[i]);
1b4         if (eq(t, 0)) return 2;
839         if (baixo == (t > eps)) qt += baixo ? 1 : -1;
d13     }
b84     return qt != 0;

```



```
1c7 }
```

3.3 Geometria Int

```
be5 struct point {
e91     int x, y;
f3e     point(int x_ = 0, int y_ = 0) : x(x_), y(y_) {}

be2     bool operator < (const point p) const {
95a         if (x != p.x) return x < p.x;
89c         return y < p.y;
679     }

418     bool operator == (const point p) const {
d74         return x == p.x and y == p.y;
a8c     }

ac5     point operator + (const point& p) const { return point(x +
p.x, y + p.y); }
6d4     point operator - (const point& p) const { return point(x -
p.x, y - p.y); }
477     template<typename U>
563     point operator * (const U c) const { return point(x * c, y *
c); }
522     auto operator * (const point& p) const { return (int)x * p.x +
(int)y * p.y; }
5db     auto operator ^ (const point& p) const { return (int)x * p.y -
(int)y * p.x; }

d70     friend istream& operator >> (istream& in, point& p) { return
in >> p.x >> p.y; }
322     friend ostream& operator << (ostream& os, const point& p) {
return os << "(" << p.x << ", " << p.y << ")"; }
252 };

b3a struct line {
b1f     point p, q;
0d6     line() {}
50a     line(point p_, point q_) : p(p_), q(q_) {}

8d7     friend istream& operator >> (istream& in, line& r) {
4cb         return in >> r.p >> r.q;
858     }

6b7     friend ostream& operator << (ostream& os, line& r) {
e8e         return os << "[" << r.p << " -> " << r.q << "];
```

```
bbb     }
7f5 };

6cb int sq(point p){
8a7     return p.x*p.x + p.y*p.y;
225 }

12e int abs(point p){
61f     return sqrt(sq(p));
b3e }

//checa se dois vetores sao perpendiculares
1f0 bool perpendicular(point v, point w){
350     return (v*w == 0);
3e1 }

//ccw, 0 colineares, -1 r a direita, 1 r a esquerda
7bf int orientation(point p, point q, point r){
c5b     int val = ((q-p) ^ (r-p));
db6     if (!val) return 0;
326     return (val > 0) ? 1 : -1;
288 }

//checa se um ponto esta contido no angulo do segmento pq e pr
5f0 bool onAngle (point p, point q, point r, point g){
999     if (!orientation(p,q,r)) return false;
673     if (orientation(p,q,r) > 0) swap(q,r);
d46     return orientation(p,q,g) >= 0 && orientation(p,r,g) <= 0;
fe6 }

// quadrante de um ponto
533 int quad(point p) {
dbb     return (p.x<0)^3*(p.y<0);
c46 }

//dist quadrada de dois pontos
4c6 int dist2(point p, point q){
f24     return sq(p.x - q.x) + sq(p.y - q.y);
507 }

// 2 * area com sinal
45e int sarea2(point p, point q, point r) {
586     return (q-p)^(r-q);
73b }

// 2 * area do poligono
c1f int polarea2(vector<point> v) {
```

```

7c9     int ret = 0;
c6e     for (int i = 0; i < v.size(); i++)
de6         ret += sarea2(point(0, 0), v[i], v[(i + 1) % v.size()]);
d03     return abs(ret);
4a3 }

//checa se o poligono e convexo
a72 bool isConvex(vector<point> p) {
ff7     bool hasPos = false, hasNeg = false;
ed0     for (int i = 0, n = p.size(); i < n; i++) {
568         int o = orientation(p[i], p[(i+1)%n], p[(i+2)%n]);
f42         if (o > 0) hasPos = true;
162         if (o < 0) hasNeg = true;
455     }
f61     return !(hasPos && hasNeg);
c83 }

a6f bool half(point p) { // true if in blue half
184     assert(p.x != 0 || p.y != 0); // the argument of (0,0)
isundefined
a12     return p.y > 0 || (p.y == 0 && p.x < 0);
326 }

//ordena os pontos de acordo com o angulo
fa6 void polarSort(vector<point> &v) {
621     sort(v.begin(), v.end(), [](point v, point w) {
8ba         return make_tuple(half(v), 0, sq(v)) <
99d         make_tuple(half(w), (v~w), sq(w));
fa2     });
68e }

//rotaciona o vetor 90 graus
5cf point rotate90(point p) {
128     return point(-p.y, p.x);
c3b }

//retorna uma linha perpendicular a linha s que passa pelo ponto p
9b2 line perpointhrough(line s, point p){
e02     return {p, p + (rotate90(s.p - s.q))}; //pode alterar a ordem
e2c }

// se p pertence ao seg de r
b49 bool isinseg (line r, point p) {
517     point a = r.p - p, b = r.q - p;
2ac     return (a ^ b) == 0 and (a * b) <= 0;
a51 }

```

```

147 bool intersect(line r, line s){
0d5     int o1 = orientation(r.p, r.q, s.p);
f92     int o2 = orientation(r.p, r.q, s.q);
033     int o3 = orientation(s.p, s.q, r.p);
885     int o4 = orientation(s.p, s.q, r.q);

bb3     if (o1 != o2 && o3 != o4) return true;

171     if (o1 == 0 && isinseg(r, s.p)) return true;
0f8     if (o2 == 0 && isinseg(r, s.q)) return true;
082     if (o3 == 0 && isinseg(s, r.p)) return true;
718     if (o4 == 0 && isinseg(s, r.q)) return true;

d1f     return false;
e86 }

// numero de pontos inteiros no segmento
9e0 int segpoints(line r) {
508     return 1 + gcd(abs(r.p.x - r.q.x), abs(r.p.y - r.q.y));
a3e }

c1f int polarea2(vector<point> v){ // 2 * area do poligono
7c9     int ret = 0;
f14     for (int i = 0; i < v.size(); i++){
de6         ret += sarea2(point(0, 0), v[i], v[(i + 1) % v.size()]);
3fe     }
d03     return abs(ret);
e97 }

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
fb6 int inpol(vector<point>& v, point p){ // 0(n)
8de     int qt = 0;
f14     for (int i = 0; i < v.size(); i++) {
bda         if (p == v[i]) return 2;
6af         int j = (i+1)%v.size();
cc6         if (p.y == v[i].y and p.y == v[j].y) {
547             if ((v[i]-p)*(v[j]-p) <= 0) return 2;
5e2             continue;
b47         }
78c         bool baixo = v[i].y < p.y;
057         if (baixo == (v[j].y < p.y)) continue;
366         auto t = (p-v[i])^(v[j]-v[i]);
2ad         if (!t) return 2;
0bb         if (baixo == (t > 0)) qt += baixo ? 1 : -1;
9cf     }

```

```

b84     return qt != 0;
604 }

421 int paralelos(vector<pair<int,int>> &pontos){
39c     int qnt = 0;
c4f     pair<int,int> e0 = reta(pontos[0], pontos[1]);
041     pair<int,int> e1 = reta(pontos[1], pontos[2]);
634     pair<int,int> e2 = reta(pontos[2], pontos[3]);
8ee     pair<int,int> e3 = reta(pontos[3], pontos[0]);
b96     if (cross(e0, e2) == 0) ++qnt;
67c     if (cross(e1, e3) == 0) ++qnt;
72a     return qnt;
eb6 }

```

4 Problemas

4.1 Centro de Árvore

```

309 vector<vector<int>> graph;
ac8 int n, dist[3][MAX];

69f void dfs(int v, int p, int id){
284     for (auto u : graph[v]){
2ab         if (u == p || dist[id][u] > 0) continue;
14a         dist[id][u] = dist[id][v] + 1;
041         dfs(u,v,id);
0ea     }
d8f }

da9 int distance(int id){
bda     int u = -1, mx = 0;
78a     for (int i = 1; i <= n; i++){
efa         if (dist[id][i] > mx){
af1             u = i;
87e             mx = dist[id][i];
da3         }
c2f     }
03f     return u;
f26 }

738 int cont(int v, int p, int d){
e40     if (d == 0) return 1;
b3d     int cur = 0;
284     for (auto u : graph[v]){
2da         if (u == p) continue;
72d         cur += cont(u,v,d-1);

```

```

075     }
75e     return cur;
2a4 }

0dd signed main () {
5d6     winton;
a68     cin >> n;
8f4     graph.resize(n+1);
6f5     for (int i = 1; i < n; i++){
ba2         int a, b;
0a8         cin >> a >> b;
8fa         graph[a].push_back(b);
4c6         graph[b].push_back(a);
080     }
c7e     dfs(1,0,0);
398     int a = distance(0);
f96     dfs(a,0,1);
924     int b = distance(1);
ee1     dfs(b,0,2);

ade     vector<int> center;
78a     for (int i = 1; i <= n; i++){
8b2         if (dist[1][i] + dist[2][i] == dist[1][b] &&
abs(dist[1][i] - dist[2][i]) <= 1){
1dc             center.push_back(i);
cef         }
7f6     }
319 }

```

4.2 Coin Collector

```

4d3 vector<vector<int>> graph, rgraph;
c7b vector<set<int>> gcomp;
7b4 stack<int> st;
daa int n, com, indeg[MAX], dp[MAX], coins[MAX], visited[MAX],
comp[MAX], gcoins[MAX];

76a void dfs(int v) {
cda     visited[v] = 1;
284     for (auto u : graph[v]){
e39         if (!visited[u]) dfs(u);
6e1     }
789     st.push(v);
850 }

664 void scc(int v, int c) {

```

```

cda    visited[v] = 1;
d5f    comp[v] = c;
af8    for (auto u : rgraph[v]){
157        if (!visited[u]) scc(u, c);
7bd    }
7e6 }

db8 void kosaraju() {
1ef    for (int i = 0; i < n; i++) visited[i] = 0;
76b    for (int i = 0; i < n; i++) if (!visited[i]) dfs(i);
1ef    for (int i = 0; i < n; i++) visited[i] = 0;
3c6    while (st.size()) {
f1a        int u = st.top();
25a        st.pop();
264        if (!visited[u]) {
e25            scc(u, com++);
1b3        }
338    }
4de }

Odd signed main(){
2e6    int m;
aa3    cin >> n >> m;
c02    graph.resize(n);
21d    rgraph.resize(n);

5ed    for (int i = 0; i < n; i++) cin >> coins[i];
dd5    for (int i = 0; i < m; i++){
ba2        int a, b;
0a8        cin >> a >> b;
9c7        graph[--a].push_back(--b);
8d3        rgraph[b].push_back(a);
a20    }
75d    kosaraju();
b2b    gcomp.resize(com);
778    for(int v = 0; v < n; v++){
284        for (auto u : graph[v]){
aa8            if (comp[u] == comp[v]) continue;

ad6            if (gcomp[comp[v]].insert(comp[u]).second){
980                indeg[comp[u]]++;
384            }
504        }
601    }
7c8    for(int i = 0; i < n; i++) gcoins[comp[i]] += coins[i];
425    for(int i = 0; i < n; i++) dp[i] = gcoins[i];
603    for(int i = 0; i < n; i++) {

```

```

ad3        debug(i, comp[i], gcoins[i], indeg[i]);
d2c    }

26a    queue<int> q;
c97    for (int i = 0; i < com; i++){
ac5        if (indeg[i] == 0) q.push(i);
21b    }

14d    while (!q.empty()){
b1e        int v = q.front();
833        q.pop();
1ed        for (auto u : gcomp[v]){
7bd            debug(u);
f27            dp[u] = max(dp[u], dp[v] + gcoins[u]);
e80            if (--indeg[u] == 0) q.push(u);
ee4        }
b12    }
1a4    int ans = 0;
603    for (int i = 0; i < n; i++){
1cf        ans = max(ans, dp[i]);
f56    }
886    cout << ans << endl;
187 }

```

4.3 Divisor Analysis

```

// quantidade, soma e produto dos divisores

bf9 int fastExpo(int base, int exp) {
c53    int r = 1;
479    base %= MOD;
fb9    while(exp) {
727        if (exp & 1) r = r * base % MOD;
a6a        base = base * base % MOD;
ef0        exp >>= 1;
a45    }
4c1    return r;
844 }

fdd int modinverse(int n) {
562    return fastExpo(n, MOD - 2);
8ff }

e9d int modiv(int a, int b){
0e2    return (a * modinverse(b))%MOD;
656 }

```

```

Odd signed main(){
1a8     int n;
a68     cin >> n;
dd9     bool par = true;
b7a     vector<int> p(n), k(n);
f40     int cnt = 1, sum = 1, prod = 1, cnt2 = 1;
603     for (int i = 0; i < n; i++){
742         cin >> p[i] >> k[i];
9d8         if (k[i]&1) par = false;
dd3     }
603     for (int i = 0; i < n; i++){
ed6         cnt = (cnt * (k[i]+1)) % MOD;
57a         cnt2 = (cnt2 * (k[i]+1)) % (2*(MOD-1));

a4a         int exp = (k[i] % (MOD-1));
db7         exp = (exp+1) % (MOD-1);
//debug(exp);
ab0         int num = (fastExpo(p[i],exp)-1 + MOD) % MOD;
27b         int den = (p[i]-1 + MOD) % MOD;
ce9         sum = (sum * modiv(num,den)%MOD)%MOD;
a41     }
603     for (int i = 0; i < n; i++){
b5b         if (!par){
bf2             int half = (cnt2/2)%(MOD-1);
220             int exp = (k[i]*half)%(MOD-1);
dc0             prod = (prod * (fastExpo(p[i],exp)%MOD)) % MOD;
b1f         }
4e6         else{
c85             int exp = ((k[i]/2)*cnt2)%(MOD-1);
dc0             prod = (prod * (fastExpo(p[i],exp)%MOD)) % MOD;
719         }
e4f     }
d2a     cout << cnt%MOD << ' ' << sum%MOD << ' ' << prod%MOD << endl;
a52 }

```

4.4 Hungaro

```

// Resolve o problema de assignment (matriz n x n)
// Colocar os valores da matriz em 'a' (pode < 0)
// assignment() retorna um par com o valor do
// assignment minimo, e a coluna escolhida por cada linha
//
// Se precisar calcular o produtorio da pra guardar o log(double(x))
// no a[i][j]
// -log se for maior produtorio e +log se for o menor produtorio

```

```
// O(n^3)
```

```

a6a template<typename T> struct hungarian{
1a8     int n;
a08     vector<vector<T>> a;
f36     vector<T> u, v;
5ff     vector<int> p, way;
f1e     T inf;

aea     hungarian(int _n, vector<vector<T>> _a){
8fd         n = _n;
92d         u.resize(n+1);
eac         v.resize(n+1);
181         p.resize(n+1);
c7c         way.resize(n+1);
ad6         a = _a;
d80         inf = INT_MAX;
c00     }

d67     pair<T, vector<int>> assignment() {
78a         for (int i = 1; i <= n; i++) {
8c9             p[0] = i;
625             int j0 = 0;
ce7             vector<T> minv(n+1, inf);
241             vector<int> used(n+1, 0);
016             do {
472                 used[j0] = true;
d24                 int i0 = p[j0], j1 = -1;
7e5                 T delta = inf;
9ac                 for (int j = 1; j <= n; j++) if (!used[j]) {
7bf                     T cur = a[i0-1][j-1] - u[i0] - v[j];
9f2                     if (cur < minv[j]) minv[j] = cur, way[j] = j0;
821                     if (minv[j] < delta) delta = minv[j], j1 = j;
4d1                 }
f63                 for (int j = 0; j <= n; j++)
2c5                     if (used[j]) u[p[j]] += delta, v[j] -= delta;
6ec                     else minv[j] -= delta;
6d4                 j0 = j1;
f4f             } while (p[j0] != 0);
016             do {
4c5                 int j1 = way[j0];
0d7                 p[j0] = p[j1];
6d4                 j0 = j1;
886             } while (j0);
38d         }
306         vector<int> ans(n);
6db         for (int j = 1; j <= n; j++) ans[p[j]-1] = j-1;

```

```

da3         return make_pair(-v[0], ans);
979     }
51f };

```

4.5 Intervalo da String Palíndromico

//Acha o maior intervalo que pode ser reordenado como palindrome numa string
//Pode ser adaptado para contar quantos palindromes e possivel fazer

```

071 int size = (1<<26);
7cf vector<int> first(size, -1);
198 int mask = 0;
b55 first[0] = 0;
87c int ans = 1;
1cf for (int i = 0; i < s.size(); i++) {
90e     int chab = s[i] - 'a';
fe3     mask ^= (1 << chab);
208     if (first[mask] != -1) ans = max(ans, (i - first[mask])+1);
033     for (int k = 0; k < 26; ++k) {
f9c         int targetmask = mask ^ (1 << k);
dc8         if (first[targetmask] != -1) ans = max(ans, (i -
first[targetmask])+1);
804     }
c26     if (first[mask] == -1) first[mask] = i+1;
b7d }

```

4.6 Inversões

```

833 int countAndMerge(vector<int>& arr, int l, int m, int r) {
fd8     int n1 = m - l + 1, n2 = r - m;
e11     vector<int> left(n1), right(n2);
bac     for (int i = 0; i < n1; i++) left[i] = arr[i + l];
d74     for (int j = 0; j < n2; j++) right[j] = arr[m + 1 + j];
5e8     int res = 0, i = 0, j = 0, k = l;
40c     while (i < n1 && j < n2) {
839         if (left[i] <= right[j]) arr[k++] = left[i++];
4e6         else {
20c             arr[k++] = right[j++];
091             res += (n1 - i);
481         }
ceb     }
694     while (i < n1) arr[k++] = left[i++];
3cc     while (j < n2) arr[k++] = right[j++];

```

```

b50     return res;
35e }

462 int countInv(vector<int>& arr, int l, int r){
11e     int res = 0;
b14     if (l < r) {
ec9         int m = (r + l) / 2;
453         res += countInv(arr, l, m);
257         res += countInv(arr, m + 1, r);
470         res += countAndMerge(arr, l, m, r);
7e6     }
b50     return res;
7d1 }

92c int inversions(vector<int> &arr) {
851     int n = arr.size();
cd9     return countInv(arr, 0, n-1);
997 }

```

4.7 Kadane

```

559 int kadane(const & vector<int> x) {
66c     int sum = x[0];
fca     int mx = x[0];
b19     for (int i = 1; i < x.size(); i++) {
cbb         sum = max(sum + x[i], x[i]);
2f2         mx = max(mx, sum);
7b0     }
b9a     return mx;
df8 }

```

4.8 Number of Pairs (GCD e LCM)

28b alguma coisa de manipular a expressao ja que o lcm(a,b) =
 $a*b/gcd(a,b)$
4e4 $x = c*(a*b/gcd(a,b)) - d*gcd(a,b)$
618 $gcd(a,b) = g$

fa2 $lcm(a,b) = g*x*y$ onde $gcd(x,y) = 1$

12c por ex se $x = 5$ e $y = 3$
3d2 $lcm(5,3) = 5*3*gcd(5,3)\{1\} = 15$
749 se a,b nao forem coprimos achamos um x,y que fazem eles serem
coprimos, de maneira que

```

3c9 a = 4
82b b = 6

9f7 lcm(a,b) = 12
20a gcd(a,b) = 2

4f8 lcm(a,b) = x*y*gcd(a,b)
701 12 = x*y*2
12d x*y = 6
587 x = 3
983 y = 2
583 ou
6d1 x = 6
6a2 y = 1

e36 lcm(4,6) = 3*2*2 correto
64f lcm(4,6) = 6*1*2 correto

578 entao lcm(a,b) = x*y*gcd(a,b)

672 x = c*x*y*g - d*g
b83 x = g((c*x*y) - d)
856 x/g = c*x*y - d
c4c */

f09 int spf[MAX], factors[MAX];

0a8 void build(){
81f     spf[1] = 1;
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
c29 }

cbc void buildfactors(){
8db     for (int i = 2; i < MAX; i++){
4b7         factors[i] = factors[i/spf[i]];
b6b         if (spf[i/spf[i]] != spf[i]) factors[i]++;
2ba     }

```

```

9cb }

c09 vector<int> getdiv(int x){
5ce     vector<int> d;
09a     for (int i = 1; i * i <= x; i++) {
72f         if (x % i == 0){
f9a             d.push_back(i);
331             if (i*i!=x) d.push_back(x/i);
7d0         }
a93     }
bae     sort(all(d));
be2     return d;
337 }

63d void solve(){
218     int c, d, x;
234     cin >> c >> d >> x;
0e8     vector<int> divisors = getdiv(x);
1a4     int ans = 0;
3fa     for (auto gc : divisors){
266         if (((x/gc)+d) % c != 0) continue;
ce4         int goal = (x/gc + d) / c;
//goal significa que eu quero achar dois caras que
//multiplicados sejam goal, e o gcd deles seja 1
//nisso eu vou descobrir a2 e b2, para achar a e b eu
//multiplico pelo u (gcd(a,b)) e calculo o lcm e depois
//testo os pares n sei como ainda
//aqui eu acho o lcm e consigo descobrir o gcd de a e b
//no caso o gcd ja eh meu divisor
//ai basta achar as combinacoes validas de a e b
//fatorar o goal e ver os fatores primos, para cada fator
//primo, a e b podem optar por receber ou nao ele
//todos os fatores primos precisam ser usados, ou em a ou
//em b
//entao temos como resposta a adicao de todas as
//combinacoes de k (qtd de fatores primos) em a,b
6fc     int k = factors[goal];
//debug(k);
741     ans += (1<<k);
495 }
886     cout << ans << endl;
450 }

0dd signed main(){
5d6     winton;
6f2     build();
d37     buildfactors();

```

```

8bd     int t;
f1b     cin >> t;
6e6     while(t--){solve();}
af8 }

```

4.9 Path Queries 2

```

2b7 #include <bits/stdc++.h>
ca4 using namespace std;

c4d using int64 = long long;
f26 const int MAXN = 200000 + 5;
c58 const int64 NEG_INF = LLONG_MIN / 4;

3b9 int n, q;
ea6 vector<int> adj[MAXN];
be6 int64 val[MAXN];

1a0 int parentArr[MAXN], depthArr[MAXN], heavy[MAXN], head[MAXN],
    pos[MAXN], szArr[MAXN];
2b0 int curPos;
2e5 vector<int64> baseArr;

383 struct SegTree {
060     int N;
5e1     vector<int64> st;
940     void init(int n) {
7b2         N = 1;
ca2         while (N < n) N <= 1;
9f7         st.assign(2*N, NEG_INF);
b1d     }
242     void build(const vector<int64>& a) {
b3b         init((int)a.size());
d50         for (int i = 0; i < (int)a.size(); ++i) st[N+i] = a[i];
d0b         for (int i = N-1; i >= 1; --i) st[i] = max(st[2*i],
st[2*i+1]);
b88     }
5b5     void update(int idx, int64 value) {
c94         int p = N + idx;
f6b         st[p] = value;
8b8         p >>= 1;
d08         while (p) {
629             st[p] = max(st[2*p], st[2*p+1]);
8b8             p >>= 1;
f03         }
007     }

```

```

889     int64 queryRange(int l, int r) { // inclusive l,r
4df         if (l > r) return NEG_INF;
a29         l += N; r += N;
cc5         int64 res = NEG_INF;
3d5         while (l <= r) {
762             if (l & 1) res = max(res, st[l++]);
059             if (!(r & 1)) res = max(res, st[r--]);
f69             l >>= 1; r >>= 1;
7a7         }
b50         return res;
500     }
974 } seg;

// first dfs: compute sizes and heavy child
a5b int dfs(int u, int p) {
81a     parentArr[u] = p;
d21     depthArr[u] = (p == -1 ? 0 : depthArr[p] + 1);
650     szArr[u] = 1;
8b7     int maxSub = 0;
032     heavy[u] = -1;
914     for (int v : adj[u]) if (v != p) {
c7d         int sub = dfs(v, u);
442         if (sub > maxSub) {
431             maxSub = sub;
3ac             heavy[u] = v;
1ab         }
a11         szArr[u] += sub;
ae1     }
0b6     return szArr[u];
64c }

// decompose: assign head and positions, fill baseArr
c88 void decompose(int u, int h) {
36a     head[u] = h;
19f     pos[u] = curPos++;
b8b     baseArr[pos[u]] = val[u];
eee     if (heavy[u] != -1) decompose(heavy[u], h);
876     for (int v : adj[u]) if (v != parentArr[u] && v != heavy[u]) {
e27         decompose(v, v);
0b4     }
950 }

249 int64 query_path(int a, int b) {
cc5     int64 res = NEG_INF;
2da     while (head[a] != head[b]) {
6d0         if (depthArr[head[a]] < depthArr[head[b]]) swap(a,b);
64e         int ha = head[a];

```



```

611         res = max(res, seg.queryRange(pos[ha], pos[a]));
0af         a = parentArr[ha];
77b     }
bb0     if (depthArr[a] < depthArr[b]) swap(a,b);
424     res = max(res, seg.queryRange(pos[b], pos[a]));
b50     return res;
ac9 }

e8d int main() {
55a     cin >> n >> q;
bae     for (int i = 0; i < n; ++i) {
c12         cin >> val[i];
b1d         adj[i].clear();
4c8     }
96b     for (int i = 0; i < n-1; ++i) {
417         int a,b; cin >> a >> b; --a; --b;
fab         adj[a].push_back(b);
c87         adj[b].push_back(a);
1b6     }
c91     dfs(0, -1);
5ad     curPos = 0;
6ea     baseArr.assign(n, NEG_INF);
70b     decompose(0, 0);
56c     seg.build(baseArr);

1f4     while (q--) {
452         int type; cin >> type;
821         if (type == 1) {
54a             int s; int64 x; cin >> s >> x; --s;
aa9             val[s] = x;
95e             seg.update(pos[s], x);
10a         } else {
417             int a,b; cin >> a >> b; --a; --b;
949             int64 ans = query_path(a,b);
cb9             cout << ans << '\n';
192         }
21b     }
bb3     return 0;
033 }

```

4.10 Steiner Tree

```

// steiner: retorna o peso da menor arvore que cobre os vertices S
// get_steiner: retorna o valor minimo e as arestas de uma solucao
// se nao tiver solucao retorna LINF
//

```

```

// grafo nao pode ter pesos negativos
// se so tiver peso nas arestas/vertices pode deletar os vw/w no codigo
//
// k = |S|
//  $O(3^k * n + 2^k * m \log m)$ 

// otimizacao: joga um vertice x do S fora e pegue a resposta em
// dp[...][x] e reconstrua a arvore a partir dele
// ta comentado no codigo as mudancas necessarias

1a8 int n; // numero de vertices
c0d vector<pair<int, int>> g[MAX]; // {vizinho, peso}
920 ll d[1 << K][MAX]; // dp[mask][v] = arvore minima com o
// subconjunto mask de S e o vertice v
0b0 ll vw[MAX]; // peso do vertice

c8f ll steiner(const vector<int> &S) {
934     int k = S.size(); // k--;
dea     for (int mask = 0; mask < (1 << k); mask++) for (int v = 0; v <
n; v++) d[mask][v] = LINF;
6b8     for (int v = 0; v < n; v++) d[0][v] = vw[v];
254     for (int i = 0; i < k; ++i) d[1 << i][S[i]] = vw[S[i]];
042     for (int mask = 1; mask < (1 << k); mask++) {
b5b         for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
638             int b = mask ^ a;
6bf             if (b > a) break;
84d             for (int v = 0; v < n; v++)
2e6                 d[mask][v] = min(d[mask][v], d[a][v] + d[b][v] -
vw[v]);
2ab         }
88c         priority_queue<pair<ll, int>> pq;
778         for (int v = 0; v < n; v++) {
6ad             if (d[mask][v] == LINF) continue;
5ca             pq.emplace(-d[mask][v], v);
f2e         }
265         while (pq.size()) {
a25             auto [ndist, u] = pq.top(); pq.pop();
dad             if (-ndist > d[mask][u]) continue;
c38             for (auto [idx, w] : g[u]) if (d[mask][idx] >
d[mask][u] + w + vw[idx]) {
679                 d[mask][idx] = d[mask][u] + w + vw[idx];
a2e                 pq.emplace(-d[mask][idx], idx);
07e             }
de5         }
a65     }
478     return d[(1 << k) - 1][S[0]]; // S[k]
//para pegar o custo minimo para juntar todos os S's basta

```

```

    percorrer a mask-1 para todo no e ver qual o menor
704 }

d41 #warning se k=1 a solucao eh a folha isolada e a funcao retorna
    edg = {}
d41 #warning se k=0 crasha
d41 pair<ll,vector<pair<int,int>>> get_steiner(const vector<int> &S) {
934     int k = S.size(); // k--;
8ec     ll ans = steiner(S);
c8d     vector<pair<int,int>> edg;
57f     stack<pair<int,int>> stk;
09b     stk.emplace((1 << k) - 1, S[0]); // S[k]
07d     while (!stk.empty()) {
c37         bool cont = 0;
9c6         auto [mask,u] = stk.top();stk.pop();
de2         if ((__builtin_popcount(mask) == 1 and u ==
S[__bit_width(mask) - 1])) continue;
851         for (auto [idx, w] : g[u]){
bb4             if (d[mask][u] == d[mask][idx] + w + vw[u]) {
fc7                 edg.emplace_back(u, idx);
8ab                 stk.emplace(mask, idx);
a04                 cont = true;
c2b                 break;
342             }
ed9         }
3b5         if (cont) continue;
b5b         for (int a = (mask - 1) & mask; a; a = (a - 1) & mask) {
638             int b = mask ^ a;
1e8             if (d[mask][u] == d[a][u] + d[b][u] - vw[u]) {
be8                 stk.emplace(a, u);
c0a                 stk.emplace(b, u);
a04                 cont = true;
c2b                 break;
f52             }
d29         }
c5c         assert(!mask || cont);
2b1     }
be8     return {ans, edg};
cf6 }

```

4.11 Subtree Queries

```

5c1 const int MAXN = 2e5+7;

fff int tree[4*MAXN], tin[MAXN], tout[MAXN], flat[MAXN];
309 vector<vector<int>>graph;

```

```

2dc int timer = 0;

955 void dfs(int v, int p){
237     tin[v] = ++timer;
284     for (auto u : graph[v]){
58d         if (u!=p){
412             dfs(u, v);
bb2         }
ab6     }
a66     tout[v] = timer;
03d }

7a2 int build(int node, int l, int r){
893     if (l == r){
019         return tree[node] = flat[l];
f94     }
ee4     int m = (l+r)/2;
174     return tree[node] = build(node*2, l, m) + build(node*2+1, m+1,
r);
97f }

d42 int query (int node, int l, int r, int a, int b){
d5e     if (b < l || r < a) return 0;
14b     if (l >= a && r <= b) return tree[node];
ee4     int m = (l+r)/2;
5a3     return query(node*2, l, m, a, b) + query(node*2+1, m+1, r, a,
b);
287 }

27f void update(int node, int l, int r, int i, int x){
791     if (i < l or i > r) return;
893     if (l == r) {
d6d         tree[node]=x;
505         return;
b74     }
ee4     int m = (l+r)/2;
3b2     update(node*2, l, m, i, x);
29a     update(node*2+1, m+1, r, i, x);
a16     tree[node] = tree[node*2] + tree[node*2+1];
d72 }

0dd signed main(){
3b9     int n, q;
55a     cin >> n >> q;
979     vector<int>arr(n+1);
78a     for (int i = 1; i <= n; i++){
5d3         cin >> arr[i];

```

```

370     }
8f4     graph.resize(n+1);
6f5     for (int i = 1; i < n; i++){
ba2         int a, b;
0a8         cin >> a >> b;
8fa         graph[a].push_back(b);
4c6         graph[b].push_back(a);
080     }
b23     dfs(1,1);
78a     for (int i = 1; i <= n; i++){
a0b         flat[tin[i]] = arr[i];
616     }
be6     build(1,1,n);

1f4     while(q--){
ffd         int op;
f99         cin >> op;
e55         if (op == 1){
d5f             int idx, valor;
066             cin >> idx >> valor;
931             update(1, 1, n, tin[idx], valor);
fcf         }
4e6         else {
8ab             int s;
a18             cin >> s;

16b             int ans = query (1,1,n,tin[s], tout[s]);
886             cout << ans << endl;
97e         }

d0f     }
a7b }

```

5 Matemática

5.1 Combinatoria

```

3e4 const int MOD = 1e9+7;

bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
479     base %= MOD;
39a     while (exp > 0) {
dea         if (exp % 2 == 1) res = (res * base) % MOD;

```

```

aa5         base = (base * base) % MOD;
727         exp /= 2;
1e2     }
b50     return res;
ab3 }

c22 int modInverse(int n) {
562     return fastExpo(n, MOD - 2);
c4f }

76f int combinations(int n, int k) {
    // Basic edge cases
032     if (k < 0 || k > n) {
bb3         return 0;
edf     }
b62     if (k == 0 || k == n) {
6a5         return 1;
f14     }
dd2     if (k > n - k) {
1b5         k = n - k;
b13     }
c92     int res = 1;
64b     for (int i = 1; i <= k; ++i) {
e56         res = res * (n - i + 1) / i;
ad4     }
b50     return res;
ff6 }

34a int combinations_mod(int n, int k) {
032     if (k < 0 || k > n) {
bb3         return 0;
edf     }
b62     if (k == 0 || k == n) {
6a5         return 1;
f14     }
c44     if (k > n / 2) {
1b5         k = n - k;
b06     }
bd9     int numerator = 1;
b3d     for(int i = 0; i < k; i++) {
f70         numerator = (numerator * (n - i)) % MOD;
020     }

cd4     int denominator = 1;
db5     for(int i = 1; i <= k; i++) {
060         denominator = (denominator * i) % MOD;
1de     }

```

```

37d     return (numerator * modInverse(denominator)) % MOD;
1e8 }

```

5.2 EV Máximo

```

/*
799 (r/m)^n - ((r-1)/m)^n = probabilidade de ter exatamente r como o
    maior resultado
b85 somatorio de P(r) * r = EV do maximo

e08 n: numero de eventos
9cb r: resultado
0ee m: quantidade de resultados possiveis para um evento
c4c */

```

```

6e6 r * ((fastExpo((r/k), n)) - (fastExpo((ld)(r-1)/k, n)));

```

5.3 Exponenciação Rápida

```

bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
fb9     while(exp) {
8ca         if (exp & 1) res = res * base % MOD;
a6a         base = base * base % MOD;
ef0         exp >>= 1;
635     }
542     return res%MOD;
2e3 }

```

```

bf9 int fastExpo(int base, int exp) {
c92     int res = 1;
fb9     while(exp) {
13e         if (exp & 1) res = res * base;
6c1         base = base * base;
ef0         exp >>= 1;
319     }
b50     return res;
4fe }

```

```

83b int fastExpo(int base, int exp, int m) {
c92     int res = 1;
054     base %= m;
fb9     while(exp) {

```

```

970         if (exp & 1) res = res * base % m;
103         base = base * base % m;
ef0         exp >>= 1;
268     }
17a     return res%m;
1b8 }

```

```

ff4 ld fastExpo(ld base, int exp) {
9d0     ld res = 1;
fb9     while(exp) {
13e         if (exp & 1) res = res * base;
6c1         base = base * base;
ef0         exp >>= 1;
319     }
b50     return res;
b1b }

```

5.4 Fatoração Prima

```

4ae vector<int> factor(int n) {
fd3     vector<int> primes;
d91     for (int p = 2; p * p <= n; ++p) {
03e         while (n % p == 0) {
bce             primes.push_back(p);
f4a             n /= p;
6e7         }
4ac     }
f3d     if (n > 1)
ccc         primes.push_back(n);
a20     return primes;
8de }

```

5.5 GCD e LCM

```

ba6 int gcd(int a, int b) {
d65     return b == 0 ? a : gcd(b, a % b);
4c5 }

```

```

3ec int mmc(int a, int b) {
8b8     return a / gcd(a, b) * b;
4c5 }

```

5.6 Phi Function

```
//quantos numeros sao coprimos com N de 1 ate N
633 int phi[MAX];
```

```
0ae void precomputePHI(){
104     for (int i = 0; i < MAX; i++)
ccf         phi[i] = i;

8db     for (int i = 2; i < MAX; i++) {
e32         if (phi[i] == i) {
d87             for (int j = i; j < MAX; j += i)
a9b                 phi[j] -= phi[j] / i;
0cc         }
f9d     }
159 }
```

5.7 Segmented Sieve

```
576 vector<char> segmentedSieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
4b2     long long lim = sqrt(R);
537     vector<char> mark(lim + 1, false);
d2d     vector<long long> primes;

b74     for (long long i = 2; i <= lim; ++i) {
c9d         if (!mark[i]) {
d38             primes.emplace_back(i);
1e8             for (long long j = i * i; j <= lim; j += i)
148                 mark[j] = true;
181         }
920     }

f9d     vector<char> isPrime(R - L + 1, true);
867     for (long long i : primes){
f8c         for (long long j = max(i * i, (L + i - 1) / i * i); j <=
R; j += i){
0d5             isPrime[j - L] = false;
cd3         }
24a     }
86a     if (L == 1) isPrime[0] = false;

    //para pegar os primos basta fazer i + L no vetor de isprime
3c8     return isPrime;
a43 }
```

5.8 Sieve

```
fd3 vector<int> primes;

8e0 void sieve() {
a94     vector<bool> is_prime(MAX + 1, true);
428     is_prime[0] = false;
8fc     is_prime[1] = false;
31f     for (int p = 2; p * p <= MAX; ++p) {
29c         if (is_prime[p]) {
305             for (int multiple = p * p; multiple <= MAX; multiple
+= p) {
4f8                 is_prime[multiple] = false;
16d             }
354         }
d56     }
e65     for (int i = 2; i <= MAX; ++i) {
4a3         if (is_prime[i]) {
e74             primes.push_back(i);
34c         }
b9d     }
ee0 }
```

5.9 Smallest Prime Factor

```
b32 int n, spf[MAX];

0a8 void build(){
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
5ce }

826 vector<int> factorize(int x){
fd3     vector<int> primes;
40e     while (x > 1){
80a         int p = spf[x];
bce         primes.push_back(p);
15a         while(x%p == 0) x/=p;
```

```

64d     }

a20     return primes;
0f8 }

//Numero de fatores primos diferentes que fatoram cada numero
11f int spf[MAX], pf[MAX];

0a8 void build(){
81f     spf[1] = 1;
ce4     for (int i = 2; i < MAX; i+=2) spf[i] = 2;
7b9     for (int i = 3; i < MAX; i+=2){
e8b         if (spf[i] == 0){
cdc             spf[i] = i;
37e             for (int j = i; j*i < MAX; j+=2){
9fe                 if (spf[i*j] == 0) spf[i*j] = i;
2bf             }
859         }
8e9     }
c29 }

da9 void buildspf(){
8db     for (int i = 2; i < MAX; i++){
e09         pf[i] = pf[i/spf[i]];
a28         if (spf[i/spf[i]] != spf[i]) pf[i]++;
3a0     }
d85 }

```

6 DP

6.1 Edit Distance

```

ac1 string s, t;
fee cin >> s >> t;
3e6 int n = s.length();
32a int m = t.length();
78a for (int i = 1; i <= n; i++){
cbc     for (int j = 1; j <= m; j++){
c70         if(s[i-1] == t[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
6d7         else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9e1     }
4b0 }

```

6.2 LCS

```

ac1 string s, t;
fee cin >> s >> t;
3e6 int n = s.length();
32a int m = t.length();
78a for (int i = 1; i <= n; i++){
cbc     for (int j = 1; j <= m; j++){
c70         if(s[i-1] == t[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
6d7         else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9e1     }
4b0 }

```

7 Grafo

7.1 Bridge Tree

```

9fa int t, c, low[MAX], pre[MAX], visited[MAX], comp[MAX];
7b4 stack<int> st;
375 vector<int> graph[MAX], compgraph[MAX];

955 void dfs(int v, int p){
cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
30c     st.emplace(v);
284     for (auto u : graph[v]){
264         if (!visited[u]){
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);
77d         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
11d     }
cf1     if (low[v] == pre[v]){ //achamos uma bridge
6a4         c++;
2f4         int cur;
016         do {
596             cur = st.top();
25a             st.pop();
524             comp[cur] = c;

```

```

cca      compgraph[c].push_back(cur);
583      } while(cur != v);
b04      }
6fc }

```

7.2 Dijkstra

```

861 int dist[MAX]
d9c void dijkstra(int st){
fe1     priority_queue<pair<int,int>> pq;
260     for (int i = 0; i < n; i++) dist[i]=LLONG_MAX;
d89     dist[st] = 0;
56c     pq.push({0,st});
502     while(!pq.empty()){
115         auto [d, v] = pq.top();
842         d = -d;
716         pq.pop();
dd8         if (d != dist[v]) continue;
940         for (auto [u,w] : graph[v]){
ac8             int novadistancia = d + w;
bd8             if (novadistancia < dist[u]) {
d96                 dist[u] = novadistancia;
030                 pq.push({-novadistancia, u});
c98             }
379         }
133     }
6f3 }

```

7.3 DSU

```

d56 struct DSU {
8f3     vector<int> comp, sz;
c1b     DSU(int n) : comp(n), sz(n, 1) {
f7a         iota(all(comp), 0);
576     }
151     int find(int u) {
dc3         if (u == comp[u]) return u;
a07         return comp[u] = find(comp[u]);
65f     }
c3b     void merge(int a, int b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
afa         sz[a] += sz[b];
ca8         comp[b] = a;

```

```

8a6         return true;
2a4     }
971 };

```

7.4 HLD - aresta

```

// SegTree de soma
// query / update de soma das arestas
//
// Complexidades:
// build - O(n)
// query_path - O(log^2 (n))
// update_path - O(log^2 (n))
// query_subtree - O(log(n))
// update_subtree - O(log(n))

// namespace SegTree { ... }

a70 namespace HLD {
af8     vector<pair<int,int>> graph[MAX];
480     int timer, pos[MAX], sz[MAX], sobe[MAX], ancestor[MAX],
        h[MAX], v[MAX];

c2a     void build_hld(int node, int p = -1, int f = 1){
f20         v[pos[node] = timer++] = sobe[node];
57c         sz[node] = 1;
50f         for (auto &i : graph[node]){
dd2             auto [u, w] = i;
2da             if (u == p) continue;
828             sobe[u] = w;
df7             ancestor[u] = node;
112             h[u] = (i == graph[node][0] ? h[node] : u);
b79             build_hld(u, node, f);
e0e             sz[node] += sz[u];

996             if (sz[u] > sz[graph[node][0].first] ||
graph[node][0].first == p){
8c8                 swap(i, graph[node][0]);
ee5             }
2b5         }
469         if (p*f == -1) build_hld(h[node] = node, -1, timer = 0);
0a5     }

1f8     void build(int root = 0){
451         timer = 0;
295         build_hld(root);

```

```

cbf      SegTree::build(timer, v);
517      }

f04      int query_path(int a, int b){
2d5          if (a == b) return 0;
aa1          if (pos[a] < pos[b]) swap(a,b);
186          if (h[a] == h[b]) return SegTree::query(pos[b] + 1,
pos[a]);
587          return SegTree::query(pos[h[a]], pos[a]) +
query_path(ancestor[h[a]], b);
cd0      }

920      void update_path(int a, int b, int x){
d54          if (a == b) return;
aa1          if (pos[a] < pos[b]) swap(a,b);

1eb          if (h[a] == h[b]) return (void)SegTree::update(pos[b]+1,
pos[a], x);
046          SegTree::update(pos[h[a]], pos[a], x);
ab0          update_path(ancestor[h[a]], b, x);
10e      }

3e4      int query_subtree(int a){
b9f          if(sz[a] == 1) return 0;
f97          return SegTree::query(pos[a]+1, pos[a] + sz[a]-1);
ce2      }

acc      void update_subtree(int a, int x){
a5a          if(sz[a] == 1) return;
84f          SegTree::update(pos[a]+1, pos[a] + sz[a]-1, x);
bf2      }

7be      int lca(int a, int b){
aa1          if (pos[a] < pos[b]) swap(a,b);
0f9          return h[a] == h[b] ? b : lca(ancestor[h[a]], b);
263      }
ea8 }

```

7.5 Kosaraju

```

881 int n, com;
042 vector<int> g[MAX];
58d vector<int> gi[MAX];
981 vector<int> g_scc[MAX];
c5a int vis[MAX];
ee6 stack<int> S;

```

```

a52 int comp[MAX];

1ca void dfs(int k) {
59a     vis[k] = 1;
7b1     for (int i = 0; i < g[k].size(); i++)
8d5         if (!vis[g[k][i]]) dfs(g[k][i]);

58f     S.push(k);
0a1 }

436 void scc(int k, int c) {
59a     vis[k] = 1;
52c     comp[k] = c;
309     for (int i = 0; i < gi[k].size(); i++)
bf6         if (!vis[gi[k][i]]) scc(gi[k][i], c);
b43 }

db8 void kosaraju() {
991     for (int i = 0; i < n; i++) vis[i] = 0;

158     for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

991     for (int i = 0; i < n; i++) vis[i] = 0;
d32     while (S.size()) {
70b         int u = S.top();
7de         S.pop();
1d6         if (!vis[u]) {
fd4             com++;
7d8             scc(u, com);
35b         }
96d     }
f84 }

// funcao para construir o grafo de componentes
ad1 void build_component_graph() {
687     for (int u = 0; u < n; u++) {
4d5         for (int v : g[u]) {
1ec             if (comp[u] != comp[v]) {
6d1                 g_scc[comp[u]].push_back(comp[v]);
5d9             }
b1e         }
1fb     }

//remove duplicatas
002     for (int i = 1; i <= com; i++) {
0a2         sort(g_scc[i].begin(), g_scc[i].end());
4ad         g_scc[i].erase(unique(g_scc[i].begin(), g_scc[i].end()),
g_scc[i].end());

```



```

6bd     }
382 }

```

7.6 LCA Sparse Table

```

/*
172 Fazer um euler tour na arvore e inserir todas as vezes que
    visitamos um no
6e7 Junto disso inserir a depth desse no em outro vetor
9f5 no final, o intervalo entre a e b sera representado por todos os
    nos que visitamos no caminho entre a e b
55b basta pegar o o no com altura minima nesse intervalo
31f para saber o ponto incial desse intervalo, basta pegar o tin de a
    e b

6e5 build O(N LOG N)
8fc query O(1)

b0d posso pensar em otimizar como chamo o lca guardando a ultima vez
    que o no foi chamado
c4c */

4f6 int n, timer = 0, tin[MAX], depth[MAX];
ba0 vector<int> et;
227 pair<int,int> sp[MAX*2][LOG+1];
309 vector<vector<int>> graph; //0 indexado

955 void dfs(int v, int p){
8c5     tin[v] = et.size();
490     et.push_back(v);
284     for (auto u : graph[v]){
58d         if (u!=p){
cf4             depth[u] = depth[v]+1;
412             dfs(u,v);
490             et.push_back(v);
f82         }
7ac     }
6e0 }

07b void buildtable(){
96d     for (int i = 0; i < (int)et.size(); i++){
1e7         sp[i][0] = {depth[et[i]], et[i]};
bcc     }
2ae     for (int j = 1; j <= LOG; j++){
538         for (int i = 0; i + (1<<(j-1)) < (int)et.size(); i++){
d30             sp[i][j] = min (sp[i][j-1], sp[i+(1<<(j-1))][j-1]);

```

```

985     }
fd8     }
641 }

2b4 pair<int,int> query (int a, int b){
bfb     int len = b - a + 1;
5e4     int lg = 31 - __builtin_clz(len);
a57     return min(sp[a][lg], sp[b - (1<<lg) + 1][lg]);
11c }

7be int lca(int a, int b){
fb2     if (tin[a] > tin[b]) swap(a,b);
ce1     return query(tin[a], tin[b]).second;
940 }

0dd signed main() {
5d6     winton;
45f     int q;
55a     cin >> n >> q;
c02     graph.resize(n);
f50     for (int i = 0; i < n-1; i++){
ba2         int a, b;
0a8         cin >> a >> b;
9c7         graph[--a].push_back(--b);
4c6         graph[b].push_back(a);
601     }
564     dfs(0,0);
df6     buildtable();
1f4     while(q--){
ba2         int a, b;
0a8         cin >> a >> b;
450         a--;b--;
092         int mac = lca(a,b);
389         cout << depth[a] + depth[b] - 2*depth[mac] << endl;
e88     }

1e3 }

```

7.7 Minimum Spanning Tree

```

d56 struct DSU {
8f3     vector<int> comp, sz;

c1b     DSU(int n) : comp(n), sz(n, 1) {
f7a         iota(all(comp), 0);
576     }

```

```

151     int find(int u) {
dc3         if (u == comp[u]) return u;
a07         return comp[u] = find(comp[u]);
65f     }

8bf     bool merge(int a, int b) {
605         a = find(a), b = find(b);
5c2         if (a == b) return false;
afa         sz[a] += sz[b];
ca8         comp[b] = a;
8a6         return true;
6bc     }
33f };

```

```

63d void solve(){
14e     int n, m;
aa3     cin >> n >> m;
73b     vector<tuple<int,int,int>> edges;
370     DSU dsu(n);
8f8     edges.resize(m);
dd5     for (int i = 0; i < m; i++){
684         int a, b, w;
060         cin >> a >> b >> w;
0d4         edges[i] = {w,a,b};
261     }
5c1     sort(all(edges));
704     int cost = 0;
dd5     for(int i = 0; i < m; i++){
13d         auto [w,u,v] = edges[i];
d19         if (dsu.merge(u,v)){
45f             cost += w;
b64         }
ce9     }
16f }

```

7.8 Pontos de Articulação

```

/*
a7e Acha os articulation points do grafo
c4c */

f7d int t, c, low[MAX], pre[MAX], visited[MAX];
864 vector<int> graph[MAX], ans;

955 void dfs(int v, int p){

```

```

cda     visited[v] = 1;
98b     low[v] = pre[v] = ++t;
14e     bool art = false;
236     int filhos = 0;
284     for (auto u : graph[v]){
264         if (!visited[u]){
701             filhos++;
412             dfs(u,v);
c80             low[v] = min(low[v], low[u]);

b71             if(low[v] >= pre[v]) art = true;
4d8         }
4e6         else {
2da             if (u == p) continue;
39b             low[v] = min(low[v], pre[u]);
acd         }
f4f     }
ca1     if (v == 1 && filhos >= 2) ans.push_back(v);
021     if (v != 1 && art) ans.push_back(v);
7fd }

```

7.9 Toposort

```

0ad int indeg[MAX];

55b vector<int> order;
26a queue<int> q;

519 for (int i = 0; i < MAX; i++){
ac5     if (indeg[i] == 0)q.push(i);
3fd }

14d while(!q.empty()){
69f     int cur = q.front();
ba5     order.push_back(cur);
833     q.pop();
411     for (auto u : graph[cur]){
472         indeg[u]--;
f90         if (indeg[u] == 0)q.push(u);
014     }
f6a }

```

8 Extra

8.1 debug.cpp

```
//Debug

/*mais simples de todos, imprime desformatado na linha*/
void dbg_out() { cerr << endl; }
template<typename H, typename... T>
void dbg_out(H h, T... t) { cerr << ' ' << h; dbg_out(t...); }

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) cerr << #__VA_ARGS__ << ':';
    dbg_out(__VA_ARGS__);
#else
#define winton ios_base::sync_with_stdio(false), cin.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*debug com cor (longo mas mt forte, imprime qualquer container
tirando coisa com .pop() -> adicionar depois)*/
namespace dbg {
    const char* const RESET      = "\033[0m";
    const char* const BOLD_BLUE  = "\033[1;34m";
    const char* const YELLOW     = "\033[33m";
    const char* const BOLD_WHITE = "\033[1;37m";

    template<typename T1, typename T2>
    ostream& operator<<(ostream& os, const pair<T1, T2>& p) { return
        os << '{' << p.first << ", " << p.second << '}''; }

    template<typename T_container, typename T = typename
        enable_if<!is_same_v<T_container, string> &&
        !is_same_v<T_container, string_view>, typename
        T_container::value_type>::type>
    ostream& operator<<(ostream& os, const T_container& v) {
        os << '{';
        bool first = true;
        for (const T& x : v) { os << (first ? "" : ", ") << x, first =
            false; }
        return os << '}'';
    }
}
```

```
}

void debug_out(string_view) { cerr << endl; }
template<typename H, typename... T>
void debug_out(string_view s, H h, T... t) {
    auto cpos = s.find(',');
    cerr << YELLOW << s.substr(0, cpos) << RESET << " = ";
    cerr << BOLD_WHITE << h << RESET;
    if constexpr (sizeof...(t) > 0) {
        cerr << ", ";
        auto nx = s.find_first_not_of(" \t\n\r", cpos + 1);
        debug_out(s.substr(nx), t...);
    } else {
        cerr << endl;
    }
}
}

using namespace dbg;

#define DEBUG

#if defined(DEBUG)
#define winton (void)0
#define debug(...) cerr << BOLD_BLUE << "[" << __func__ << ":" <<
    __LINE__ << "]" << RESET << " "; debug_out(__VA_ARGS__,
    __VA_ARGS__)
#else
#define winton
    ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
#define debug(...) (void)0
#endif
//=====//

/*debug mais simples, so imprime vars mas imprime organizado na linha,
menor e mais facil de codar*/
void debug_out(string s, int line) { cerr << endl; }
template<typename H, typename... T>
void debug_out(string s, int line, H h, T... t){
    do{
        cerr << line << ": " << s[0]; s = s.substr(1);
    }
    while (sz(s) and s[0] != ',');
    cerr << " = " << h;
    debug_out(s, line, t...);
}
```

```

#define DEBUG

#if defined(DEBUG)
    #define winton (void)0
    #define debug(...) debug_out(#__VA_ARGS__, __LINE__, __VA_ARGS__)
#else
    #define winton ios_base::sync_with_stdio(false), cin.tie(NULL)
    #define debug(...) (void)0
#endif
//=====//

/*debug foda mas sem cor*/
namespace dbg {
    template<typename T1, typename T2>
    ostream& operator<<(ostream& os, const pair<T1, T2>& p) { return
        os << '{' << p.first << ", " << p.second << '}''; }

    template<typename T_container, typename T = typename
        enable_if<!is_same_v<T_container, string> &&
        !is_same_v<T_container, string_view>, typename
        T_container::value_type>::type>
    ostream& operator<<(ostream& os, const T_container& v) {
        os << '{';
        bool first = true;
        for (const T& x : v) { os << (first ? "" : ", ") << x, first =
            false; }
        return os << '}'';
    }

    void debug_out(string_view) { cerr << endl; }
    template<typename H, typename... T>
    void debug_out(string_view s, H h, T... t) {
        auto cpos = s.find(',');
        cerr << s.substr(0, cpos) << " = " << h;
        if constexpr (sizeof...(t) > 0) {
            cerr << ", ";
            auto nx = s.find_first_not_of(" \t\n\r", cpos + 1);
            debug_out(s.substr(nx), t...);
        } else {
            cerr << endl;
        }
    }
}

using namespace dbg;

#define DEBUG

```

```

#if defined(DEBUG)
    #define winton (void)0
    #define debug(...) cerr << "[" << __func__ << ":" << __LINE__ <<
        "]" << " "; debug_out(#__VA_ARGS__, __VA_ARGS__)
#else
    #define winton
        ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
    #define debug(...) (void)0
#endif
//=====//

/*generic printer de todas as estruturas*/
// Generic printer for vector<T>
template <typename T>
ostream &operator<<(ostream &os, const vector<T> &v) {
    for (size_t i = 0; i < v.size(); ++i) {
        os << v[i];
        if (i + 1 < v.size()) os << ' ';
    }
    return os;
}

// Specialized printer for vector<vector<T>> (matrix)
template <typename T>
ostream &operator<<(ostream &os, const vector<vector<T>> &matrix) {
    for (size_t i = 0; i < matrix.size(); ++i) {
        os << matrix[i];
        if (i + 1 < matrix.size()) os << '\n';
    }
    return os;
}

//-----PAIR-----//
template<class A, class B> ostream& operator<<(ostream& os, const
    pair<A, B>& p) {
    os << "(" << p.first << ", " << p.second << ")";
    return os;
}

//-----VECTOR-----//
template<typename T> ostream& operator<<(ostream& os, const vector<T>&
    vec) {
    os << "[ ";
    for(const auto& elem : vec) {
        os << elem << " ";
    }
}

```

```

    }
    os << "]" ;
    return os;
}
//-----SET-----//
template<typename T> ostream& operator<<(ostream& os, const set<T>& s)
{
    os << "{ ";
    for(const auto& elem : s) {
        os << elem << " ";
    }
    os << "}";
    return os;
}
//-----MULTISET-----//
template<typename T> ostream& operator<<(ostream& os, const
multiset<T>& s) {
    os << "{ ";
    for(const auto& elem : s) {
        os << elem << " ";
    }
    os << "}";
    return os;
}
//-----QUEUE-----//
template<typename T> ostream& operator<<(ostream& os, queue<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.front() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----DEQUEUE-----//
template<typename T> ostream& operator<<(ostream& os, deque<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.front() << " ";
        q.pop_front();
    }
    os << "}";
    // Print a newline at the end
    return os;
}

```

```

}
//-----STACK-----//
template<typename T> ostream& operator<<(ostream& os, stack<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.top() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----PRIORITY_QUEUE-----//
template<typename T> ostream& operator<<(ostream& os,
priority_queue<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.top() << " ";
        q.pop();
    }
    os << "}";
    // Print a newline at the end
    return os;
}
//-----MAP-----//
template<typename K, typename V> ostream& operator<<(ostream& os,
const map<K, V>& m) {
    os << "{ ";
    for(const auto& pair : m) {
        os << pair.first << " : " << pair.second << ", ";
    }
    os << "}";
    return os;
}

template<typename T>
using min_pq = priority_queue<T, vector<T>, greater<T>>;
template<typename T> ostream& operator<<(ostream& os, min_pq<T> q) {
    // Print each element in the queue
    os << "{ ";
    while (!q.empty()) {
        os << q.top() << " ";
        q.pop();
    }
    os << "}";
}

```

```

    // Print a newline at the end
    return os;
}

```

8.2 Template.cpp

```

//Template
#include <bits/stdc++.h>
using namespace std;

#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define endl '\n'
#define int long long
#define ld long double

void dbg_out() { cerr << endl; }
template<typename H, typename... T>
void dbg_out(H h, T... t) { cerr << ' ' << h; dbg_out(t...); }

#define DEBUG

#if defined(DEBUG)
    #define bg3 (void)0
    #define debug(...) cerr << #__VA_ARGS__ << ':';
        dbg_out(__VA_ARGS__);
#else
    #define bg3 ios_base::sync_with_stdio(false), cin.tie(NULL)
    #define debug(...) (void)0
#endif

signed main(){
    bg3;
}

```